Begin with the name of Allah



Group Members:

Qaisar Khan(50850)
Hamza Ahmed(50861)
Abdul Jabbar(50843)

Supervisor Name:

Mr. Junaid Sultan

A project Submitted in fulfillment of the requirement of Diploma Associate Engineering (Electronics)

1st/OCT/2024

Acknowledgement

We are deeply grateful to Almighty Allah, without whose guidance and blessings this project would not have been possible. We would like to extend our sincere thanks to our mentors, particularly Director SITT(Sir Ahson Ansari) , DH Electronics(Sir Sohail Ahmed), Sir Danish-ul-Khairi, and Sir Umair Siddique, for granting us the opportunity to undertake this project. We are especially indebted to Sir Haseeb and his group for their invaluable assistance with Image Processing, as well as to our supervisor, Mr. Junaid Sultan, for his unwavering support and insightful guidance throughout the project. Our heartfelt appreciation also goes to the Mechanical Department and the SETC PCB Lab for their constructive support , and to all those who contributed to the troubleshooting efforts. Your support was instrumental in the completion of this work.

DOCUMENTATION PAGE

TITLE: Project Report Remote Sensing Communication Can-Sat	**DOCUMENT DATE and EDITION:** OCT. 2024, 1st Ed.
AUTHOR: Qaisar Khan (50850) Abdul Jabbar (50843) Hamza Ahmed (50861)	**Supervisor**: Junaid Sultan

KEYWORDS: Can-Sat,

ABSTRACT:

This Document outlines the design, requirements and parts selection for building a Remote Sensing Communication Can-Sat, termed as Can-Sat which is able to acquire the environmental data such as Temperature, Pressure, Altitude, Direction of heading, Longitude etc and transmit it in real-time, it makes a soft landing with a constant velocity of 4.5 m/s due to the compact parachute design, it also acquires images of the environment and transmit it to the Ground Station. The Satellite is developed under the rules of 10th Iran Can-Sat Competition 2024, the mission aims at data acquisition and image processing by OpenCV using C++ in the field of agriculture, environment and crisis management. The Can-Sat is designed around dimensions, mass requirements and mission requirements as set in competition guidelines. This can be a miniature Can-shaped satellite to simulate the upcoming SUPARCO missions on ground level.

Table of Contents

Acknowledgement .....ب

1-1) Team Organization 1
2) Introduction 2
(1-2 Can-Sat overview 2
(3 Mission Definition and Objectives 3
3-1) Mission Statement 3
Main Objectives: 3
Bonus Objective: 3
(2-3 Concept of Operations (CONOPS) 3
3-3) Requirements and their implementation 5
System requirements 6
Subsystem requirements 7
Communication & Data Handling (CDH) Sub System Requirements 9
Electrical Power System (EPS) Requirements 11
Mechanical Structure Requirements 12
Descent Control / Recovery System Requirements 13
Ground Control System Requirements 13
4) Literature Review 14
4) System designs Overview 15
System Design 1: 15
Main Features: 15
System Design 2: 17
Main Features: 17
Main Features: 19
Main Features: 19
(1-6 System and subsystem architecture and interactions 23
System architecture 23
The Can-Sat Container Contains following subsystems 23
Ground Control System contains 24
Recovery Sub System Design 24
Sensor Sub System Design 25
Sensor Subsystem Specifications 26
Mechanical Sub System Design 27

Communication & Data Handling (CDH) Sub System Design 28
Data Handling and Control 28
Communication Sub System Design 28
Electrical Power System (EPS) Design 30
Battery: 30
Power switch: 30
Power indicator: 30
Batter voltage sense: 30
Distribution system: 30
Ground Control System (GCS) Design 31
GCS Software 31
Telemetry 32
Interactions between the subsystems and components 32
(6-2 Theoretical and scientific Foundation of the design 33
6-3) Test 36
(6-1 Scheduling 37
5)Budgeting 38
7)References 39
8)conclusion 40
9)Appendix 40
Coding 40
6.1) ESP32SUPERMINI: 40
6.2) ESP32CAM: 42
6.3 Ground control system 57

List of Figures

Title **Page**

Figure 2-1: Can-Sat Mission Profile 5
Figure 3-1 Outer Design 16
Figure 3-2 Outer Design 16
Figure 3-3 internal sub-system 17
Figure 3-4 Design 17
Figure 3-6internal PCB 18
Figure 3-7 Design view 18
Figure 3-8 final Design view 19
Figure 3-9 open assembly 20
Figure 3-10 closed assembly 21
Figure 3-11 Can-Sat final design and labeled assembly 22
Figure 3-12 our fully assembled Can-Sat 25
Figure 3-13 mechanical sub-system design 27
Figure 3-14 esp32 wroom pinout 28
Figure 3-15 esp32 supermini pinout 28
Figure 3-16 lora module 29
Figure 3-17 Interfaces of electronic and communication subsystems in a Can-Sat 30
Figure 3-18 GCS block diagram 31
Figure 3-19 Graphic user interface (GUI) 31
Figure 3-20 Interfaces of electronic and communication subsystems in a Can-Sat 33
Figure 3-21 flight test 37
Figure 3-22 our fully assembled Can-Sat 34

List of Tables

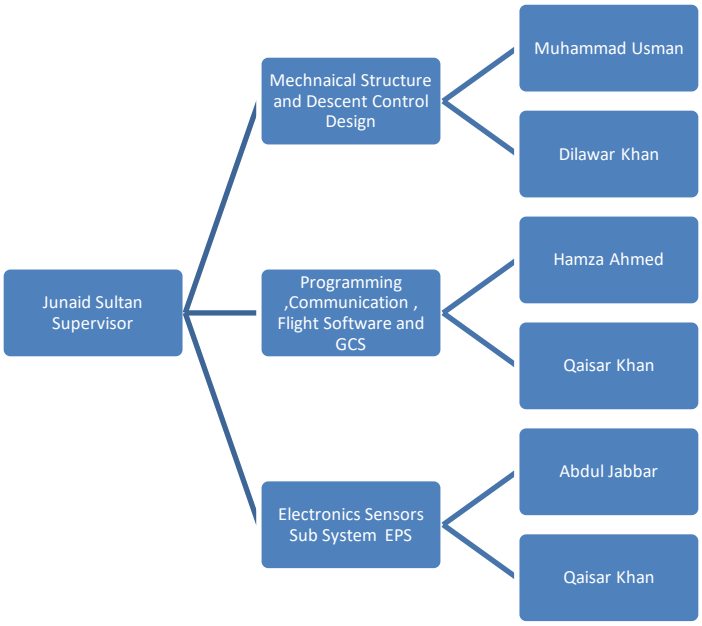
Title **Page**

Table 1-2: mission requirements 4
Table 2-3 Table of Requirements for the sensor sub-System 7
Table 2-4 Table of Requirements for the communication System 9
Table 2-5 Table of Requirements for the EPS System 11
Table 2-6 Table of Requirements for mechanical structure 12
Table 2-7:Recovery System Requirements 13
Table 2-8:GCS Requirements of Can-Sat 13

Abbreviations

Can-Sat	Can Satellite
EPS	Electrical power system
GCS	Ground control system
GUI	Graphical user interface
PLA	Polylactic acid
PDR	Preliminary design report
CDR	Critical design report
GPS	Global positioning system
I2C	Inter integrated circuit
UART	Universal asynchronous receiver transmitter
NMEA	National marine electronics association string
CDH	Communication data handling
ESP 32	Espressif Systems Protocol 32
Tx	Transmit
Rx	Receive
CSV	Comma separated values

1-1) **Team Organization**



2) Introduction

2-1) **Can-Sat overview**

A **Can-Sat** is a miniature representation of a real satellite, cleverly integrated within the volume and shape of a **soft drink Can** (typically a 330 ml can). The challenge for students is to fit all the major subsystems found in a satellite—such as power, sensors, and a communication system—into this compact space. The Can-Sat is then launched to an altitude of a few hundred meters by a rocket or dropped from a Drone or captive balloon and its mission begins: to carry out a scientific experiment and achieve a safe landing.

Can-Sat offer a unique opportunity for students to have a first practical experience of a real space project. They are responsible for all aspects: designing the Can-Sat, selecting its mission, integrating the components, testing, preparing for launch and then analyzing the data.







Satellite → Can → Can-Satellite CanSat

3) Mission Definition and Objectives

3-1) **Mission Statement**

The mission of the Remote Sensing-Communication Can-Sat is to measure environmental data such as temperature, altitude, and pressure, as well as to take images and send them to the ground station. The mission defined in this competition is data acquisition and image processing by artificial intelligence(openCV) in the field of agriculture, environment and crisis management using c++. In this mission, Can-Sat is released over ground targets and it must measure the area of target and detecting their colors by using image processing algorithms. In this regard, detecting targets and their colors must be applied in a semi-automatic or automatic manner

Main Objectives:

1) Design and Build a Can-Sat to be launched and deployed from a drone at an altitude of about 300 meters. The Can-Sat is to descend at 5 m/s or less. It is to meet all the requirements and constraints as laid down in table of requirement at Table 2-2
2) Can-Sat is to measure environmental data such as temperature, humidity, and pressure.
3) Can-Sat must transmit GPS data, sensors data, and Can-Sat health status information to the ground in real time
4) Can-Sat is released over ground targets and it must measure the area of target and detecting their colors by using image processing algorithms. In this regard, detecting targets and their colors must be applied in a semi-automatic or automatic manner.
5) The structure must fulfill dimensions and mass requirements of competitions as specified in table of requirement at Table: 2-2.
6) The Can-Sat landing mechanism must be designed in such a way that it lands the payload at the closest point to the ground target area.

Bonus Objective:

Using the Internet platform to exchange data between the Can-Sat and the ground station in each stage of the mission (optional) will have extra scores depending on the quality of the work.

3-2) **Concept of Operations (CONOPS)**

The Can-Sat competition scenario as follows:

1. Can-Sat is turned on and placed in releaser. Releaser is used for ascending and releasing Can-Sat and it is provided by competition organizers
2. The ascending Can-Sat collects mission data such as temperature, pressure, and so on and transmits them to the Ground Station in real-time.
3. Can-Sat is released after reaching the specified altitude of the competition. Can-Sat must also collect and transmit data during the release phase.
4. The descending Can-Sat must capture images from desired area on the ground using its camera.
   These targets must be identified using the images and reported to the ground station during or after the flight.

The area of targets and their colors should be automatically reported by the Can-Sat.

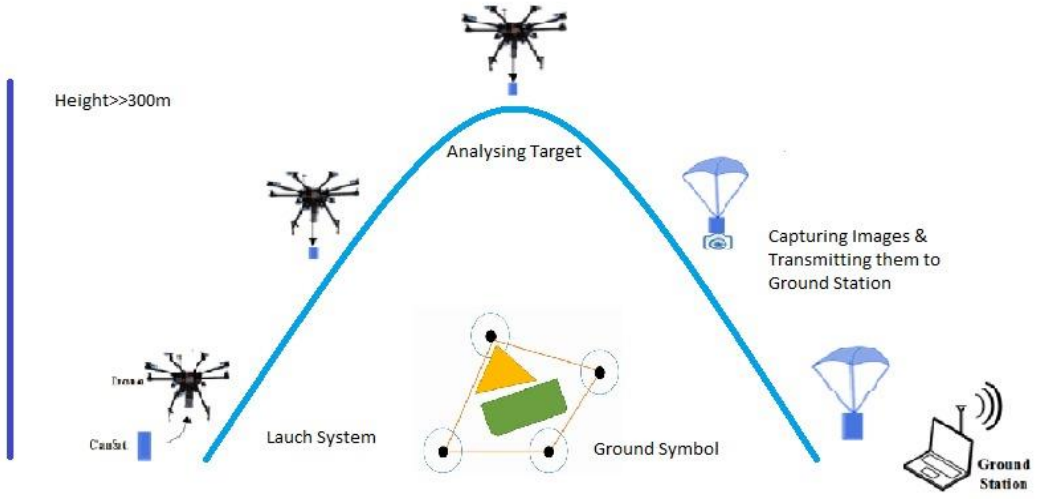
5. Can-Sat should have a safe landing using the recovery mechanism after release.
6. A safe landing area the nearest distance within the target should be considered, considering cross-wind or external chaos.
7. After reaching the target position, Can-Sat stops and declares its presence so that it can be tracked.
8. After finding Can-Sat, it is turned off and the mission is over.
9. Assessment of the results is carried out using observations by the referees choose by the Competition Organizers.

Table 1-1: mission requirements

Step	duration(s)	start time(s)	Description

Can-Sat Turned on and placed in releaser		0	Start of mission
Launch take-off	-	100	Start of flight
Can-Sat collects mission data and transmits to GCS		100	Real Time data transmission
Release from drone	-	130	
Parachute deployment for recovery	-	135-205	Landing phase using parachute
Capture and transmit images during descent	-	140-190	Take photos and data gathering
landing on earth	-	210	Beacon and GPS to find
Finding and turning off the system		270	End of Mission

Figure 3-1: Can-Sat Mission Profile



3-3) **Requirements and their implementation**

System requirements

Technical requirements of the system for competition are listed in table below:

Table 3-1: Table of Requirements for the System

Code	Requirement	Source	Physical specification	Priority	Approval method				
					Design review	Test	Analysis	Control	A/TBA
SYS.1	The mass of Can-Sat must be less than 350 gr	Competition.	Structure	high				✓	A
SYS.2	In classical class, Can-Sat must fit inside a standard soda can (115 mm height and 66 mm diameter)	Competition	Structure	high				✓	A
SYS.3	Can-Sat must be compatible with the ascending system (during ascending, acceleration changes in the design must be considered)	Competition	Structure, Recovery	high				✓	A
SYS.4	Can-Sat must land safely to the ground (conditions of descending, acceleration changes, and landing must be considered in design)	Mission	Recovery	medium		✓			A
SYS.5	The use of toxic and flammable materials is not allowed in the design of Can-Sat	Competition	Structure	medium		✓			A
SYS.6	Can-Sat must obey the RF regulation	competition	Telecommun ication	high	✓		✓		A
SYS.7	Can-Sat must measure atmospheric parameters	mission	payload	high		✓			A

SYS.8	Can-Sat must collect data related to atmospheric sensor or data in payload such as temperature	mission	Data management and command	high	✓				A
SYS.9	Can-Sat must transmit GPS data, sensors data, and Can-Sat health status information	mission	Transmissio n/telecommunication	medium	✓	✓			A
SYS.1	Can-Sat must be in operational status for 90 minutes	Operation concept	power	low		✓	✓		A
SYS.1	Can-Sat must be resistant to operation load (launching phase, phase of releasing before parachute, releasing with a parachute, and landing)	Operation concept	structure	medium		✓	✓		A
SYS.1	Each Can-Sat must be equipped with a beacon system to declare its presence at the location	mission	payload	high			✓		A

Subsystem requirements

Table 3-2 Table of Requirements for the sensor sub-System

Code	**Requirement**	**Source**	**Physical specification**	**Priority**	**Approval method**				**Status**
					Design review	**Test**	**Analysis**	**Control**	
SCDH.1	Can-Sat must measure	Mission	Data management	High		✓	✓		A

	atmospheric parameters like Air Pressure.		and command.						
SCDH.2	Can-Sat must collect data related to atmospheric sensor or data in payload such as temperature.	Mission	Data management and command.	High		✓	✓		A
SCDH.3	Can-Sat must measure atmospheric pressure to measure Altitude of Can-Sat.	Mission	Data management and command.	High		✓	✓		A
SCDH.4	Can-Sat should transmit GPS data.	Can-Sat bonus Requirement	Data management and command	Low	✓	✓		✓	A
SCDH.5	Each Can-Sat must be equipped with a beacon system to declare its presence at the location	Mission	Payload	High	✓	✓		✓	A
SCDH.6	Can-Sat must detect Enclosed area of targets and their colors (presenting the area of targets by inserting the area and color on the image).	Competition	Payload	High	✓	✓	✓	✓	A
SCDH.7	Can-Sat must measure atmospheric parameters like humidity.	Can-Sat bonus Requirement	Data management and command	Low	✓		✓		TBA
SCDH.8	Can-Sat should give Air Condition as an	Can-Sat bonus Requirement	Data management and command	Low	✓		✓		TBA

	atmospheric Parameter.								
SCDH.9	Can-Sat should give its Direction and its orientation.	Can-Sat bonus Requirement	Payload	Low	✓		✓		A

Communication & Data Handling (CDH) Sub System Requirements

Table 3-3 Table of Requirements for the communication System

Code	**Requirement**	**Source**	**Physical specification**	**Priority**	**Approval method**				Status
					Design review	**Test**	**Analysis**	**Control**	
CDH.1	Can-Sat must obey the RF regulation	Mission	Transmission/	High		✓	✓		A

CDH.2	Can-Sat must transmit GPS data, sensors data, and Can-Sat health status information to the ground in real time	Mission	tele-communication	High		✓	✓		A
CDH.3	Sending measured parameters in the requested data format immediately after the measurement	Mission	Transmission/	High		✓	✓		A
CDH.4	The communication frequency of the Can-Sat with the ground station should be in the range of amateur radio frequencies of: 430-440 MHz or 2400 to 2450 MHz	Mission	tele-communication	Low	✓	✓		✓	A
CDH.5	Transmitted power shall be less than 20 dBm (or 100 mW).	Mission	Transmission/	High	✓	✓		✓	TBA
CDH.6	Telecommunications of Can-Sat must cover a range of more than 500 meters (Intersection requirement)	Mission	tele-communication	High	✓	✓	✓	✓	A
CDH.7	Can-Sat must be able to have a one-way telecommunication with the ground station continuously (from Can-Sat to the ground station).	Mission	Transmission/	Low	✓		✓		A
CDH.8	Updating data rate at least every second	Mission	Tele-communication	Low	✓		✓		A
CDH.9	At least three images should be transmitted at the time of descent Images should be completely transmitted in digital format - The images must be compressed using compression methods	Mission	Transmission/ tele-communication	Low	✓		✓		TBA

CDH.10	if using any network-supported communications module, each team should choose a unique random NETID/PANID code. The network settings including NETID/PANID code must be reported in CDR	Mission	Transmission/ tele-communication	✓		✓	✓		TBA
CDH.11	Using the Internet platform to exchange data between the Can-Sat and the ground station in each stage of the mission (optional) will have extra scores depending on the quality of the work	Competition Bonus requirement	Transmission/ tele-communication	✓		✓	✓		NA
CDH.12	Valid and reliable telecommunication between Ground Station and Can-Sat without interference	Operational	Transmission/ tele-communication	✓		✓	✓		A
CDH.13	Can-Sat must store collected data and captured images onboard memory card and on GCS	Operational	Data Handling	✓	✓				A
CDH.14	Order of data storage should be in alpha recovery (read from left to right)	Competition requirement	Data Handling						A
CDH.15	File extensions should be .CSV or .TXT	Competition requirement	Data Handling			✓	✓		A

Electrical Power System (EPS) Requirements

Table 3-4 Table of Requirements for the EPS System

Code	Requirement	Source	Physical specification	Priority	Approval method				Status
					Design review	test	analysis	control	
EPS.1	The electrical power supply subsystem of the Can-Sat must provide the required power for the entire operation time, including the time needed to recover after landing and at least 60 minutes before the operation.	Competition	Power System	HIGH		✓	✓		A
EPS.2	Can-Sat must be in operational status for 90 minutes	Operation	Power System	HIGH		✓	✓		A
EPS.3	Can-Sat shall measure its battery Voltage	Operation	Power System	LOW	✓				A
EPS.4	Battery type and size must comply requirement constrains	Operation	Power System	HIGH	✓				A

Mechanical Structure Requirements

Table 3-5 Table of Requirements for mechanical structure

code	requirement	source	Physical specification	Priority	Approval method				Status
					Design review	test	analysis	control	
MSS.1	The mass of Can-Sat must be less than 350 gr	competition	Structure	HIGH				✓	A
MSS.2	In classical class, Can-Sat must fit inside a standard soda can (115 mm height and 66 mm diameter)	competition	Structure	HIGH				✓	A
MSS.3	Can-Sat must be compatible with the ascending system (during ascending, acceleration changes in the design must be considered)	competition	Structure, recovery	HIGH				✓	A
MSS.4	Can-Sat must land safely to the ground (conditions of descending, acceleration changes, and landing must be considered in design)	mission	Recovery	MEDIUM		✓			A
MSS.5	The use of toxic and flammable materials is not allowed in the design of Can-Sat	competition	Structure	MEDIUM		✓			A

MSS.6	The team's name shall be printed on the Can-Sat to help with the identification of the Can-Sats. Also, a colored ribbon, which is visible from far away, must be applied axially to the surface of Can-Sat.	competition	Structure	HIGH				✓	A

Descent Control / Recovery System Requirements

Table 3-6:Recovery System Requirements

Tcode	**requirement**	**source**	**Physical specification**	**priority**	**Approval method**				Status
					Design review	**test**	**analysis**	**control**	
RSS.1	The recovery subsystem must be designed with a descent rate equal to or less than 5m/s.	competition	Recovery	HIGH				✓	A
RSS.2	Can-Sat should have a safe landing using the recovery mechanism after release.	competition	Recovery	HIGH				✓	A
RSS.3	A safe landing at the nearest distance within the area of targets should be considered, should be given to Considering cross-wind or external chaos.	competition	Recovery	HIGH				✓	TBA
RSS.4	Each Can-Sat must be equipped with a beacon system to declare its presence at the location	mission	Payload, recovery	HIGH			✓		A

Ground Control System Requirements

Table 3-7:GCS Requirements of Can-Sat

code	**requirement**	**source**		**priority**	**Approval method**	Status

			Physical specification		**Design review**	**test**	**analysis**	**control**	
GCS.1	Each Team should design its own GCS	Competition Rules	Ground Control System	HIGH	✓	✓	✓	✓	A
GCS.2	The Ground Station must receive data in real time.	Mission Scenario	Ground Control System	HIGH	✓	✓	✓	✓	A
GCS.3	Directional Antenna should be used for GCS	Operational	Ground Control System	Medium	✓	✓	✓	✓	A
GCS.4	Can-Sat shall operate autonomously and no command from Ground Station is allowed to be sent during the operation scenario (telecommand is not allowed).	Competition Rules	Ground Control System	HIGH	✓	✓	✓	✓	A
GCS.5	Appropriate graphical display of received and analyzed data immediately in Ground Station	Competition Rules	Ground Control System	HIGH	✓	✓	✓	✓	A
GCS.6	Image Processing of received images for target area and specifications	Competition Rules	Ground Control System	HIGH	✓	✓	✓	✓	A

4) Literature Review

1)Introduction

The exploration of extraplanetary destinations has significantly advanced through the use of robotic systems. Historically, these devices were expensive to build and operate, often sponsored by the military or governmental institutions. However, recent developments have led to the emergence of small-scale and low-cost scientific platforms known as Can-Sats. These cylindrical-shaped devices offer an economical alternative for conducting high-risk investigations within Earth's atmosphere[1].We got to much information and knowledge from the previous Can-Sat compitations that occurred in different states like in European Space Agency etc which provided us the way to achieve our targets

2)Historical Context

Can-Sats have evolved alongside their cube-shaped counterparts, CubeSats. While CubeSats gained popularity as secondary payloads in space missions due to their small size and minimal weight, Can-Sats maintained their unique cylindrical

form. Their reduced costs and ability to operate within Earth's atmosphere make them attractive for near-space science research

5) Current Applications

Can-Sats have proven versatile in various applications:

- Earth Science: Researchers use Can-Sats for environmental monitoring, weather data collection, and studying atmospheric phenomena.
- In Compitations: Different Can-Sats are designed to win different compitations like that one we are performing The 10th International Iran Can-Sat Compitaton.

6) Potential Impact

The affordability of Can-Sats widens participation in near-space science. Moreover, their role as intermediary technologies—testing and improving space-bound hardware—positions them for future space exploration efforts. Can-Sats could play a crucial role in shaping the technology pipeline for space missions

4) System designs Overview

Multiple designs were reviewed for the system

System Design 1:

The first design aim at creating space for payload, it provides ease for assembling and testing the system.

Main Features:

1. Flat Hexagonal Parachute used which will be deployed after release at 300 meters, by using a miniature Servo
2. Flexible antenna used for omni directional transmission

3. Electronics System Consists of Separate PCBs for Sub Systems, which will be mounted separately and connected bus connectors, which creates space for components and ease in assembly
4. Raspberry Pi is mounted vertically with a rods
5. Pay Load Camera is place at center of bottom Lid
6. Battery and EPS section is placed at bottom section
7. Threaded rods are used for support
8. Outer Cover will be 3D printed and of thin width. Which will be reduce weight of the system.

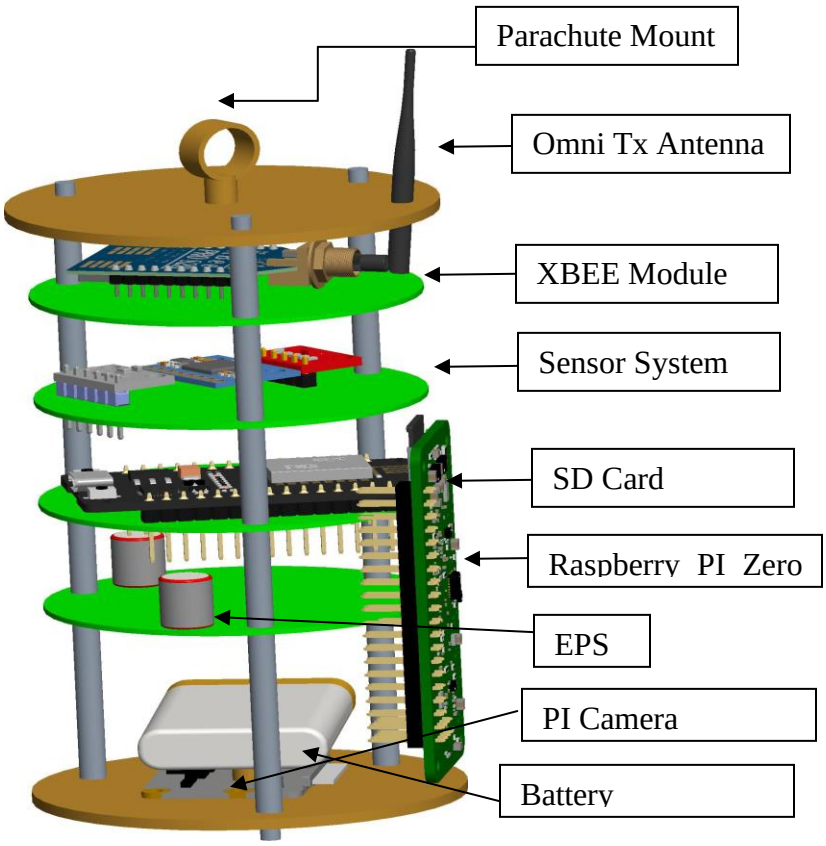


Figure 2-1 Outer Design

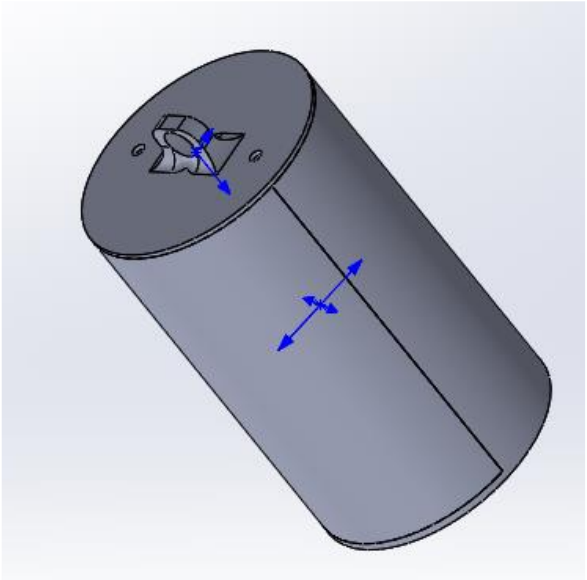


Figure 3-2 Outer Design

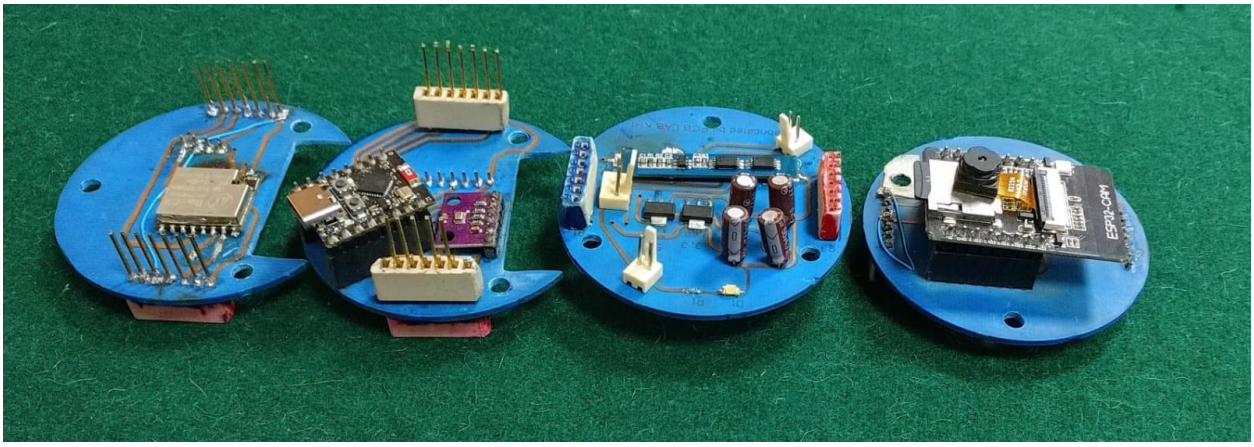


Figure 4-3 internal sub-system

System Design 2:
The first design aim at reducing weight

Main Features:

1. Flexible antenna used for omni directional transmission
2. Electronics System will be in one bay and mounted vertically through Plastic mounting sticks
3. Pay Load Camera will be mounted through a servo at bottom Lid for target alignment and stability
4. Battery and EPS section is placed at bottom section
5. Threaded rods are used for support
6. The Structure will be 3D printed
7. Flat Hexagonal Parachute used which will be deployed after release at 300 meters, by using a miniature Servo
8. No Steel rods in the system which will reduce weight
9. The Top Cover and the container are threaded assembling it easily and Parachute attachment
10. Flat Hexagonal Parachute used which will be deployed after release at 300 meters, by using a release switch.

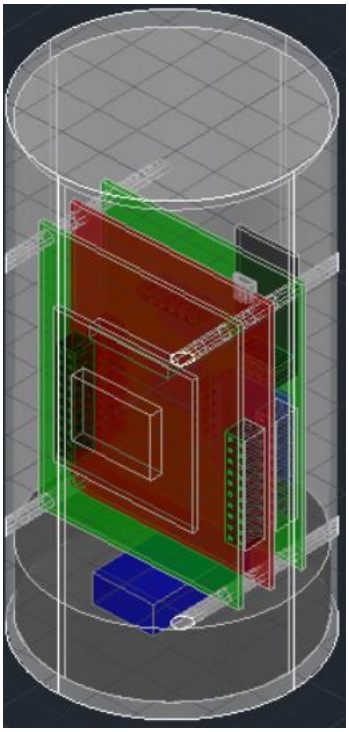


Figure 5-4 Design



Figure 3-5 3D View

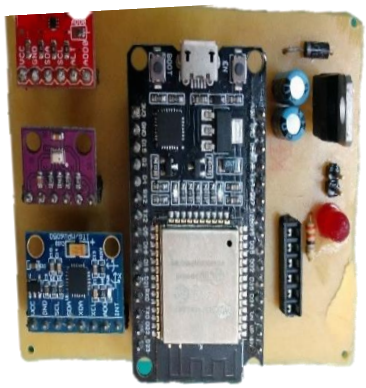


Figure 6-6internal PCB

System Design 3:

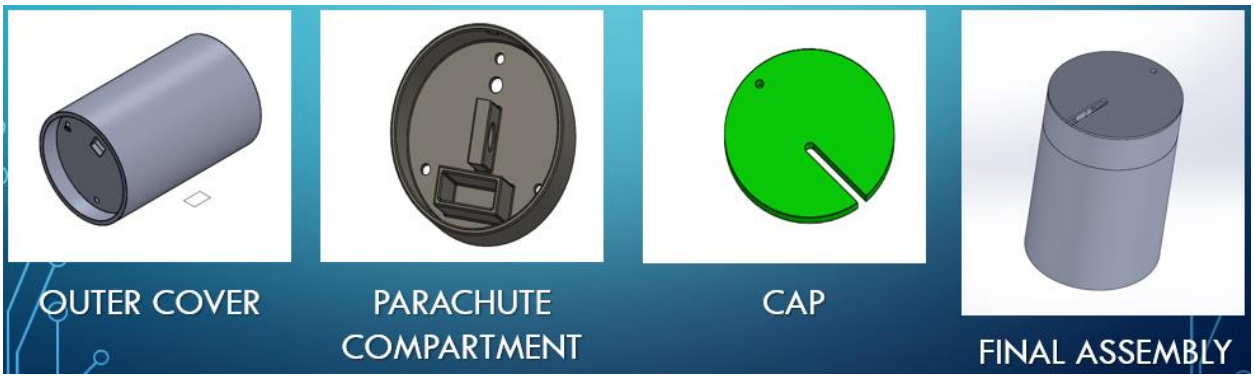


Figure 7-7 Design view

Main Features:

1) This design was solid enough for our Can-Sat .
2) It was easy to fabricate for us on the 3d printer and easily designable.
3) But it had some issues like placing the Electronic mechanism(PCBs) inside it was not easy.
4) Taking that system out of the body was not easily accessible .
5) The main issue was the connection of Can-Sat antenna with GCS antenna due to completely filled body of design.
6) We were unable to catch satellite to connect GPS antenna with them in a small period of time.
7) There was the issue related to parachute release mechanism because the parachute was unable to fit inside the compartment.
8) The last issue with it was the weight of body which was consuming 45% of Can-Sat over all weight.

Final design

System Design 4:



Figure 8-8 final Design view

Main Features:

1) This design was solid enough for our Can-Sat .
2) It was easy to fabricate for us on the 3d printer and easily designable.

3) It is a compact for our system PCBS are easily movable in & out of the body due to its two half sections.
4) The issue of antenna connection was also resolved due to the hollow shape.
5) The main issue of GPS aonnection with Satellite was resolved as intenna has no obstacle due holes in body shape.
6) There was the issue related to parachute release mechanism because the parachute was unable to fit inside the compartment now that's also resolved as we place the parachute up to the cap and control it by servo changed location.
7) The main issue of weight also resolved as its hollow and weight reduced from 45% to 30 % and easily openable/closable.

Final selection of design and internal structure:

Considering all the features, it was decided to select Design 4 , which will be provide , easy accessibility , less power consumption , more space , ease assembling ,no resistance for Antennas and GPS to provide a secure connection and safe handling of subsystems at the time of testing.



Figure 9-9 open assembly



Figure 10-10 closed assembly

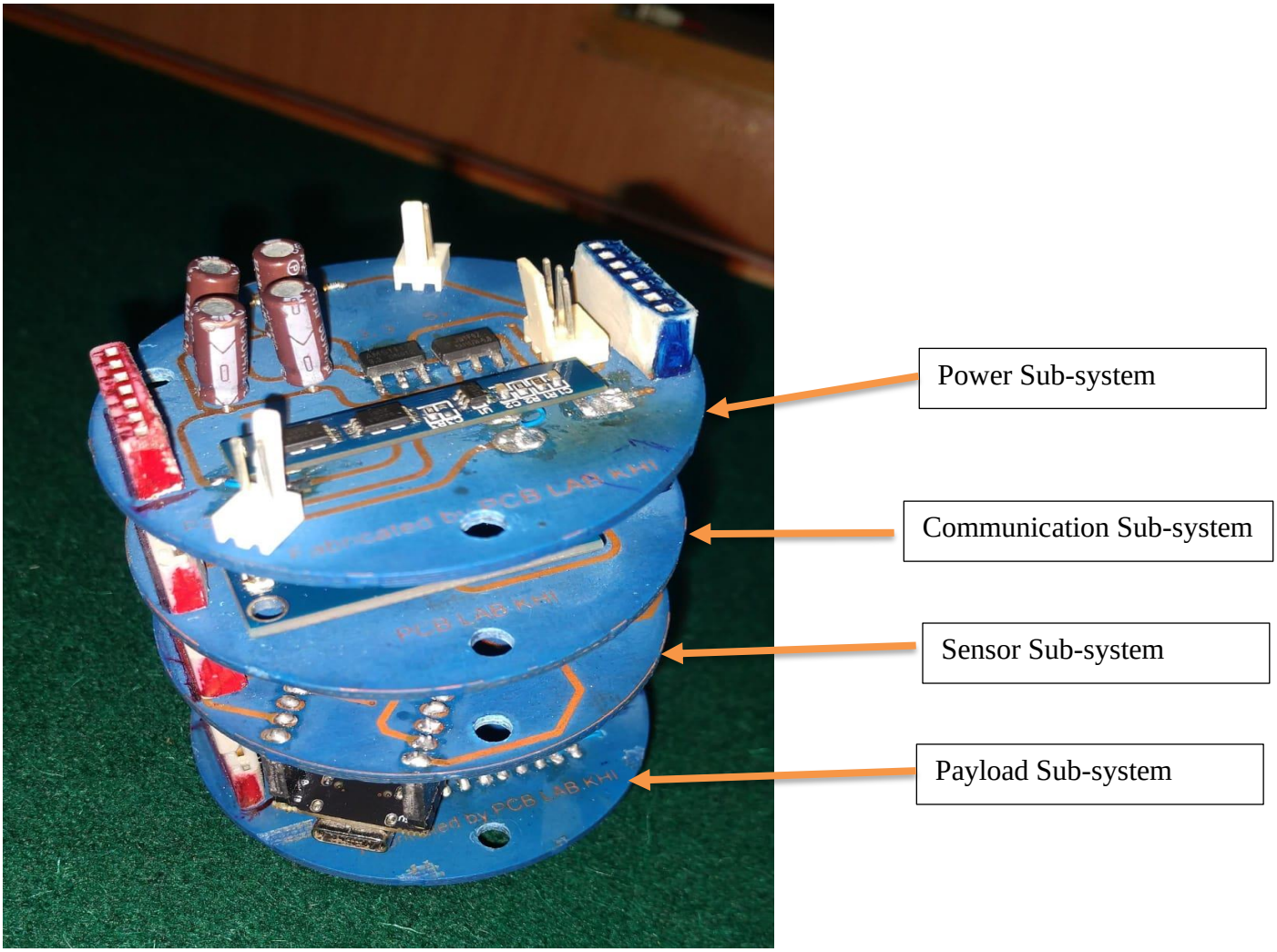


Figure 11-11 Can-Sat final design and labeled assembly

6-1) **System and subsystem architecture and interactions**

System architecture

The Whole System comprises of two units: 1. Can-Sat Container 2. Ground Control System

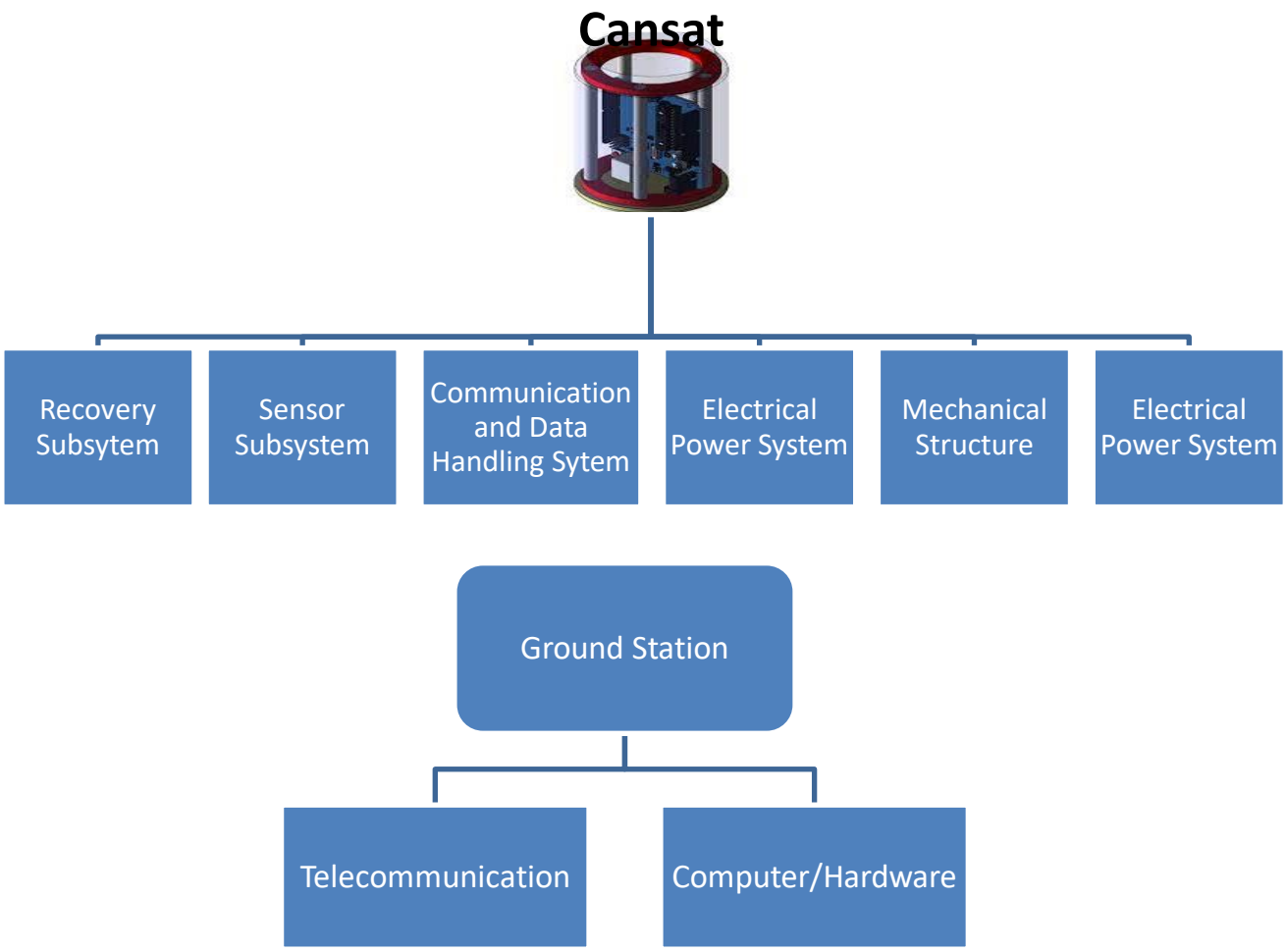


Figure 0-6: Can-Sat system architecture

The Can-Sat Container Contains following subsystems

1. Recovery Subsystem
2. Sensor Subsystem
3. Communication and Data Handling System
4. Mechanical Structure
5. Electrical Power System

Ground Control System contains

Antenna Reception System with Laptop for GUI, analysis and storage.

Recovery Sub System Design

This section should be devoted to the architecture of the subsystems. Specifications of each subsection should be clearly presented with sufficient details. For example, in accordance with Fig. 2-3, the recovery subsystem consists of the parachute subsystem, mechanical actuators, and electronic circuits to receive command signals, etc.

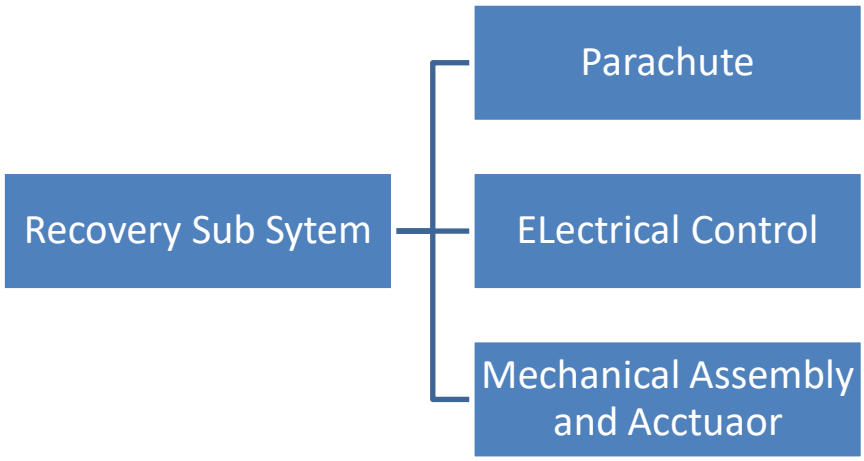


Figure 0-7: Example of architecture and configuration of Can-Sat recovery subsystem

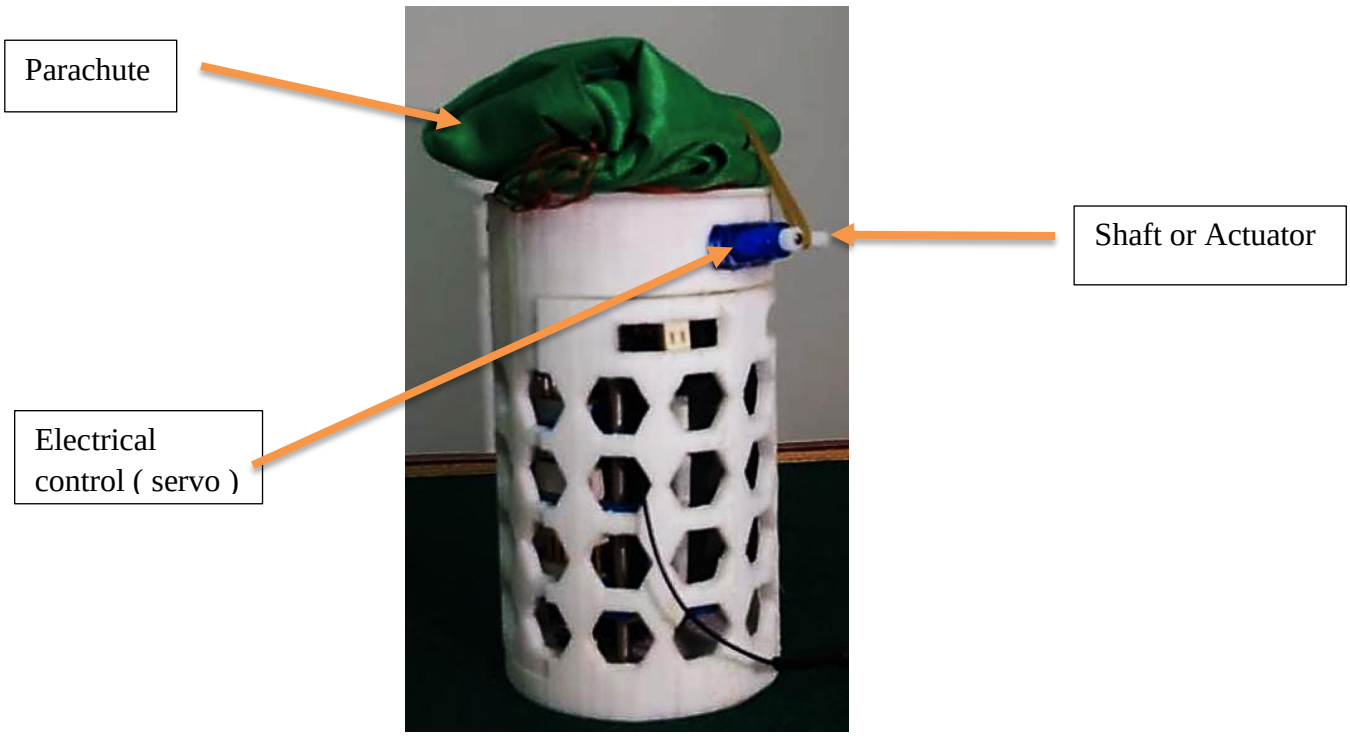


Figure 12-12 our fully assembled Can-Sat

Sensor Sub System Design

Table 0-9:GCS Requirements of Can-Sat

S.Code	Sensor Type	Selected Sensor Model	Purpose	Section
SSO.1	Air Pressure	BMP-280	Measuring Air Pressure as its mission requirement	Structure
SSO.2	Altitude/Height	BMP-280	Measuring Air Pressure to determine Can-Sat height/altitude	Structure
SSO.3	GPS	NEO-6M	Finding Local or global position of the ground targets and Specially Ground Systems (optional)	Structure/ Tethered Payload
SSO.4	Location Indicator	Buzzer	To declare the presence of Can-Sat at the location	Structure
SSO.5	Image(Area+ Color)	Esp32 CAM	To acquire and save images.	Tethered Payload
SSO.6	Compass (Direction orientation)	MPU-6050	For the smooth Control of Can-Sat Directivity and its orientation	Structure

Sensor Subsystem Specifications

Table 3-2 Specifications of Selected Sensors

SensorModel	Interface	Range (Min-Max)	Current (µA)	Volts (V)	Dimensions (HxWxL(mm))	Mass (grams)	Price ($)
BMP-280 	I2C/SPI	300 hPa-1100hPa	0.5 µA at 1 Hz	3.3v	2 X 6 X 13.5	3	3
NEO-6M 	UART	>50 Km altitude	20 to 80 mA	2.7 to 3.6	2.4 X 12.2 X 16	<10	6.11
Buzzer 	Digital	<15m	<30 mA	4 to 8	2.6 mm Diameter	2	0.49
MPU-6050 	I2C	X=±250 Y=±500 Z=±1000	3mA	1.5 to 3.6	0.9 x 4 x 4	5	1.5
ESP-32 CAM 	SPI	Res:1080 x 640 x 480 Fps:60,30 View:120°	<110 mA	3.3 to 5	.9 x 24 x 24.5	~15	5

Mechanical Sub System Design

This section should be devoted to the architecture of the subsystems. Specifications of each subsection should be clearly presented with sufficient details. For example, in accordance with Fig. 2-3, the recovery subsystem consists of the parachute subsystem, mechanical actuators, and electronic circuits to receive command signals, etc.

Figure 13-13 mechanical sub-system design



Communication & Data Handling (CDH) Sub System Design

Data Handling and Control

Microcontrollers (ESP-32 Wroom + ESP-32 supermini C3)

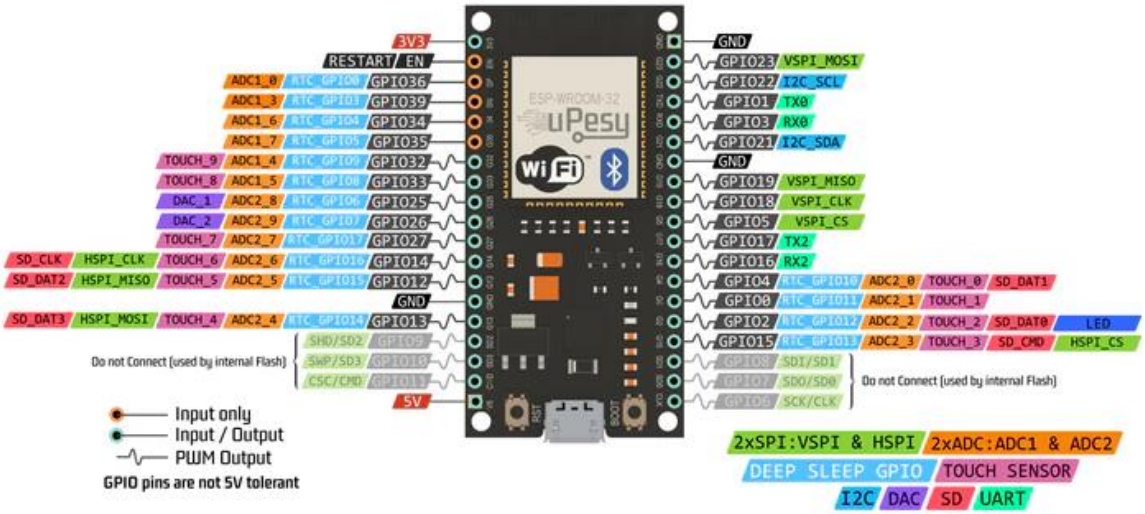


Figure 14-14 esp32 wroom pinout

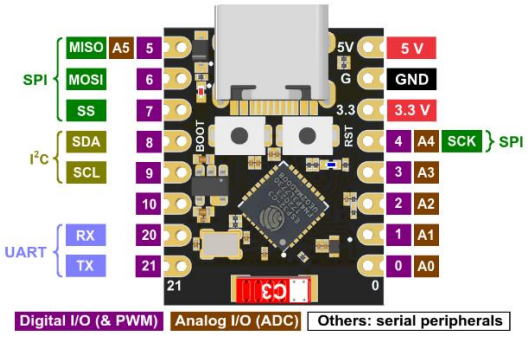


Figure 15-15 esp32 supermini pinout

Communication Sub System Design

Can-Sat to Ground Link

- Communication with ground station using long range antenna (LORA)
- Can-Sat Transmitting Data to Ground Station

Table 3-3: Trade off Communication System

Range	Transmit Power	Sensitivity	Frequency	Baud Rate	Operating current	Operating Voltage	Description
• Urban up to 2KM • Rural up to 10kM	1000 mW 20dBm	- dB 137m	ISM 433 MHz	115200 bps	Transmit Current Max output 20dBm 120mA Receive Current Single RX 0.5uA	1.8 to 3.6V	LORA MODULE SX-1278

Selection Reason:

Long range connectivity.

Low power consumption.

Low cost and simplicity



Figure 16-16 lora module

Electrical Power System (EPS) Design

Connector for charging

Battery:

It was used as power source of the components for at least two hours.

Power switch:

Diverts and interrupts the current flow from batter BMS 2 cell cal system.

Power indicator:

Indicates if the system obtains energy.

Batter voltage sense:

battery voltage sense is to monitor the electrical potential of a battery to ensure optimal performance and prevent overcharging or discharging.

Distribution system:

Guarantee that the other electronics will receive the correct voltage and current levels

for the entire time of mission.

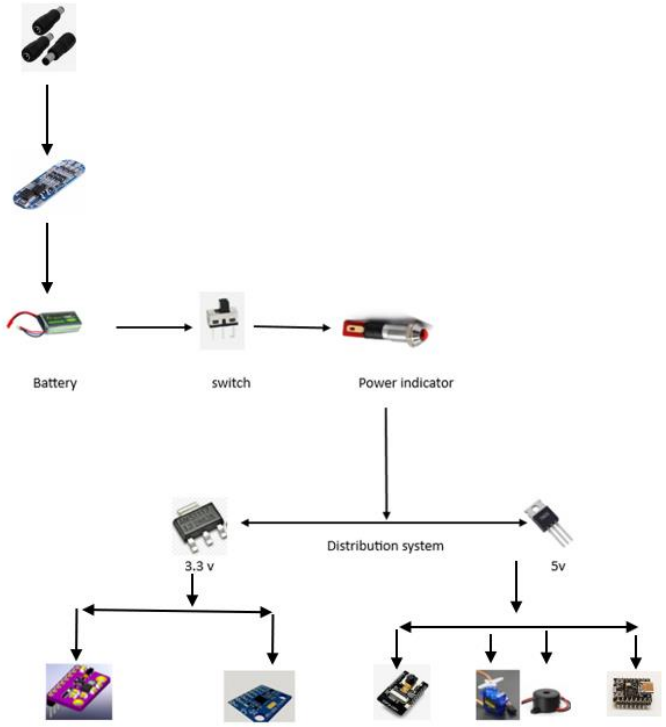


Figure 17-17 Interfaces of electronic and communication subsystems in a Can-Sat

Ground Control System (GCS) Design

- Communication with ground station using long range antenna (LORA)
- Can-Sat Transmitting Data to Ground Station using LORA module

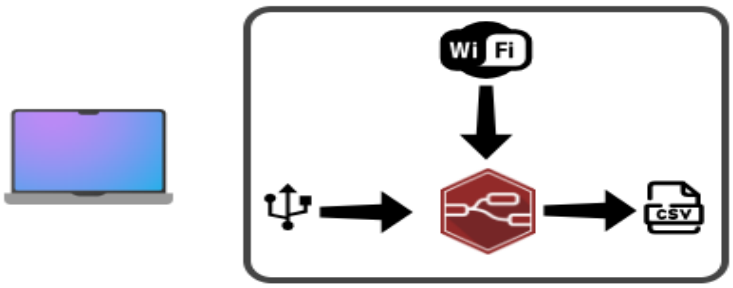


Figure 18-18 GCS block diagram

GCS Software

GCS software will be built using **Node-red or LabVIEW** as a real time plotting application. It will process the data received on the XBEE radio. It will also be responsible for either https requests or MQTT packet submission. We are not using any licensed software.

Figure 19-19 Graphic user interface (GUI)



Telemetry

The Telemetry will be saved in a CSV file format. The image files will be extracted through https requests.

CSVString Format:

packet,RTC_time,gps_latitude,gps_longitude,gps_speed,bmp_temp,bmp_pressure,bmp_altitude, roll,pitch;

01:33:45,5624,0.0000000,0.0000000,0,31.54,10042659,9.65,0.91,9.66,1.12,-0.05,0.01,0.01

01:33:46,5625,0.0000000,0.0000000,0,31.53,10042648,9.66,1.02,9.68,1.13,-0.04,0.01,0.00

01:33:47,5626,0.0000000,0.0000000,0,31.52,10426.04,9.69,0.87,9.65,1.18,-0.04,0.01,0.01

01:33:48,5627,0.0000000,0.0000000,0,31.51,10042559,9.73,0.90,9.63,1.04,-0.04,0.01,0.01

01:33:49,5628,0.0000000,0.0000000,0,31.1,100429.09,9.44,0.92,9.59,1.15,-0.04,0.01,0.02

01:33:50,5629,0.0000000,0.0000000,0,31.51,10431.59,9.23,0.89,9.70,1.15,-0.05,0.01,-0.00

01:33:51,5630,0.0000000,0.0000000,0,31.53,100436.31,.83,0.96,9.57,1.16,-0.04,0.01,0.00

01:33:52,5631,0.0000000,0.0000000,0,31.57,10043618,8.84,0.90,9.65,1.12,-0.04,0.01,0.01

01:33:53,5632,0.0000000,0.0000000,0,31.1,100434.03,9.02,0.97,9.62,1.14,-0.05,0.01,0.01

01:33:54,5633,0.0000000,0.0000000,0,31.63,10043403,9.02,0.97,9.62,1.23,-0.05,0.01,0.00

01:33:55,5634,0.0000000,0.0000000,0,31.66,100434.45,.99,0.96,9.58,1.22,-0.05,0.01,0.00

Interactions between the subsystems and components

Main Microcontroller ESP32 acquires data immediately after turning on power, stores data in onboard and memory and transmits data each second to ground station

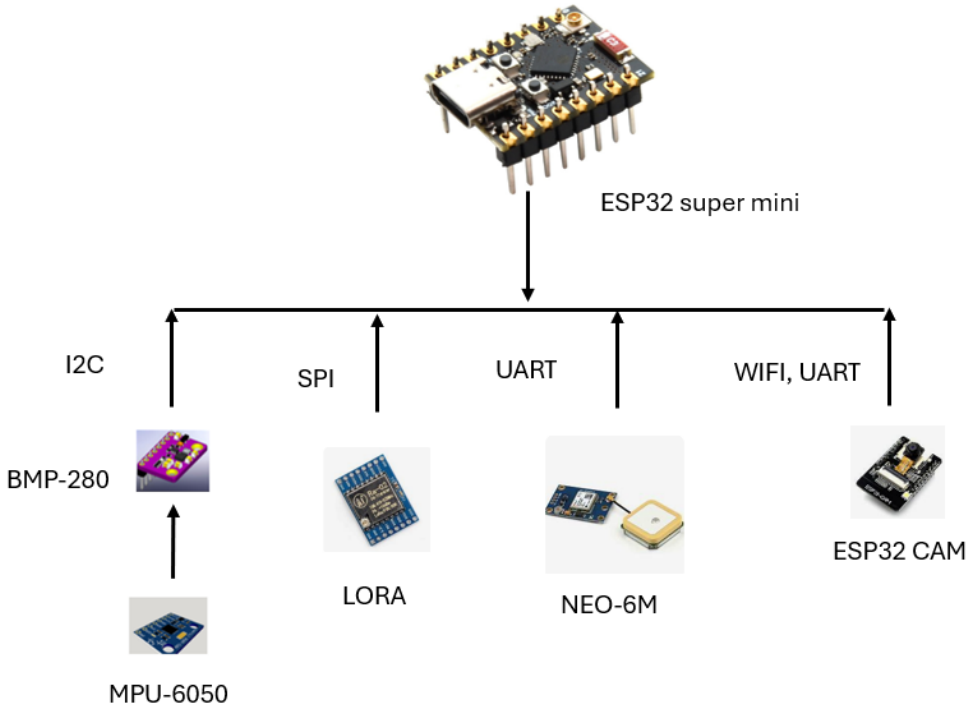


Figure 20-20 Interfaces of electronic and communication subsystems in a Can-Sat

6-2) **Theoretical and scientific Foundation of the design**

This subsection presents the scientific aspect of the proposed design from an engineering perspective. For example, the reason for using a delayed parachute, calculating the required time for delay, and supplying landing speed according to the requirement must be presented. In this section, satisfaction of requirements must be defined.

OUR DESIGN:

We have selected the following flat/hexagonal shaped parachute for our can-sat:



Figure 21-22 our fully assembled Can-Sat

WHY WE CHOSE FLAT / HEXAGON SHAPED PARACHUTE?

Following are the reasons:

Provides inherent stability during descent.

Provides precise control over descent rate of can-sat.

It can be easily folded and packed in a compact manner.

It extends duration of descent phase to provide more time for data collection.

PARACHUTE WORKING:

FG = mg

FD = 1/2 CD ρAv^2

Where,

FG = Force of Gravity

FD = Drag Force

A = Area of the parachute

CD = Co - efficient of Drag

V = Velocity of the falling object

P = Density of Gas

CALCULATION:

We are using the formula:

FG = FD

Þ mg – 0.5CD ρ Av^2 = 0

Þ A = 2mg/ (CD ρ v^2)

Þ A = pi * D^2 / 4 (for Flat Parachute

Þ (pi * D^2 / 4) = (2mg/ (CD ρ v^2)

Þ D = √ (8mg ÷ (pi ∗ CD ρ v^2)

Mass of Can-Sat = 350gms

Using 1.225 kg/m³ for the density of air

9.81 m/s² for gravity acceleration at sea level

5 m/s descent rate

0.75 for the drag of a flat parachute

Using all these values in above equation:

Þ D = √ 8 (350) (9.81) ÷ (3.1417 ∗ 1000 * 1.225 ∗ 4.5^2)

Þ D = √27468 ÷ 77933.79

Þ D = √ 0.35

Þ D = 0.591 m

Þ D = 591mm

In order to find out length of sides of hexagon we use following formula:

Þ S = √ (0.785 * D^2 ÷2.5981)

Þ S = √ (0.785 * 591^2 ÷2.5981)

Þ S = √ (273137.742 ÷2.5981)

Þ S = √105129.803

Þ S = 324.23 mm

6-3) **Test**

To confirm satisfaction of requirements, the system needs to be thoroughly tested. The space industry uses a bottom-up process for performing the test, which means that first, components are tested to fix problems, if any. After assurance of proper functioning, components are integrated to form a subsystem. In this step, each subsystem has to be tested individually to check its function. A subsystem may not be able to perform its predefined functions. This fault might be due to an Intersection of parts that can be repaired and fixed. However, in some cases, the fault may be due to design errors. In this case, the design must be modified and improved, or the subsystem must be reconstructed. The testing process is a repeatable process that can be repeated several times for a subsystem or a part to eventually achieve the desired result. System integration is carried out after approving the proper functioning of subsystems, and tests are performed on the entire system to ensure its functioning. In this section, teams must provide a specific program for the Can-Sat testing in their report. This means which parts have to be tested to approve the function and performance of Can-Sat.

Prerequisites, type of test, method of test, and how to apply results must be mentioned too.

It should be noted that the defined items of this section must be compatible with the column of "approval method" in the requirements table 2-2.



Figure 22-21 flight test

6-1) **Scheduling**

Usually, each project has several phases, such as conceptual design (feasibility), preliminary design, critical design, construction, testing, approval, operation, etc., which should be done according to the schedule at the specified time. Teams should prepare their scheduling for different phases according to the provided overall program. It can be provided in the form of a Gantt chart using software such as Microsoft Project.



5)Budgeting

System budgeting involves mass, power, data, costs, and human resource budgeting.

In preliminary design, it is necessary to determine the mass, power and data volume required for processing, transmitting, or storing and also allocate the cost of the system to each subsystem (top to bottom).

Budgeting at the critical design phase must be done for parts and components, and it should also be approved from bottom to top.

Teams should provide a table for each item of system budgeting, such as Table 2-4. In system design, a margin is usually proposed for each item of budgeting to avoid a lack of funds in case of design modification in subsequent phases or when construction and implementation errors occur.

In the preliminary design, the system is broken into subsystems (as in Table 2-4); however, in the critical design phase, the system is divided into parts, and the power consumption, mass, data volume, and cost of each one will be mentioned in budgeting.

Table 0-4: An example of a budgeting table

Subsystem	**Components**	**Mass Budget (Grams)**	**Power Budget (Watts)**	**Finance Budget (Dollars $)**	**Human Resources (Man hours)**

Sensor Subsystem	BMP-280	3	1.65µ	3	
	BUZZER	2	0.15	0.49	
	NEO-6M	9	148m	6.11	
	MPU-6050	5	66 µ	1.5	
Electrical Power System	Li ion Battery (1000mAh)	35	7	3	
	Switch	3	-	0.1	
	Switch Indicator	0.3	20m	0.2	12
	Reg(3.3v)	2	-	0.2	
	Reg(5v)	2	-	0.2	
CDH System	ESP-32 super mini	8	300m	2	
	SD Card module	2.7	0.165	0.5	26
	SD Card	1	-	1.9	
	LORA MODULE SX-1278	6	200	4	
Camera control	Servo + jig	12	1.2	3	4
Decent Control System	Servo Mechanism	5	0.48	1.5	
	Parachute	35	-	1.5	24
Mechanical Structure	Design	~50	-	13	
	SENSORs PCB	~10	-	2	
	CDH PCB	~10	-	2	
	EPS PCB	~10	-	2	
TOTAL	**-**	**260 g**	**10 w**	**~55$**	**96 hours**

7)References

If any document, website, or book is used as a reference in the report, it must be referred to. Referring to sources is mandatory while using figures and tables. Text reference can be carried out by using numbers in brackets such as [1], [1,2], [1-4]. There is no difference between English and Persian references. Writing references are as follows, which is the same for an article, book, or pamphlet.

https://www.kildarenow.com/news/confey-college-win-leinsterfinals-Can-Sat-space-competition/214097

http://dit.ie/newsandevents/news/archive2018/news/title,158988

https://17chankinsellaa.wixsite.com/confeyCan-Sat2018

8)conclusion

In conclusion, our Can-Sat project successfully demonstrated the capabilities of a miniature satellite system in a simulated space environment, achieving accurate telemetry data transmission, reliable autonomous systems, and effective sensor and camera performance. Through this project, we gained invaluable hands-on experience in design, testing, and teamwork, overcoming challenges and constraints to deliver a functional and efficient Can-Sat. The project's success highlights the potential for innovative and compact satellite solutions in aerospace, research, and industry, and we look forward to applying the knowledge and skills .acquired to future projects, driving advancements in STEM education and space exploration

9)Appendix

Coding

6.1) ESP32SUPERMINI:

```
#include "Sensors.h"                    // Custom made Library

#define SW_TX              20

#define SW_RX              21

EspSoftwareSerial::UART ss;          // Used to define swSer

HardwareSerial cam_s(0);

const char* ssid = "CAM";

const char* password = "HAMZA154657";

const char* serverUrl = "http://192.168.4.1/image.jpg";  // URL for the image

static const int RXPin = 11, TXPin = 10;

int packet = 0;

byte take_pic = 'A';

byte transmit = 'C';

byte Log = 'B';
```

```
bool first = 1;


void setup() {
  Serial.begin(115200);              // Only to be executed if debugging
  cam_s.setPins(RXPin,TXPin);       // Set Pins for RX , TX for communication with CAM
  cam_s.begin (250000);             // Initialize Serial for GPS communication
  ss.begin(9600, EspSoftwareSerial::SWSERIAL_8N1, SW_RX, SW_TX );  // Initialize
Software Serial for communication with ESP_CAM
  mpu6050_setup();                  // Initialize the mpu6050 Sensor
  bmp280_setup();                   // Initialize the bmp280 Sensor
  //bmp_180.begin();
  LORA_SETUP();                      // Initialize LoRa
  delay(500);
  RTC_setup();                      // Initialize RTC
  timer_setup();        // Initialize the timer to send an interrupt every 1 Second to raise the intr flag
}
void loop() {
// If 1 second has passed then send the string if not then update the string
if (intr == 1){
  /*
                    CSVString Format:
packet,RTC_time,gps_latitude,gps_longitude,gps_speed,bmp_temp,bmp_pressure,bmp_altitude,
roll,pitch;
  */
    get_RTC();
    CSV_STRING += packet;
    CSV_STRING += ',';
```

```
    get_gps();
    get_bmp_280();                // Add the values of bmp280 to CSV_STRING
    get_mpu6050();                // Add the values of mpu6050 to CSV_STRING
  LoRa.beginPacket();
    LoRa.print (CSV_STRING);          // Send the CSV_STRING
  LoRa.endPacket();
  CSV_STRING += 'E';
  cam_s.write(Log);
  cam_s.write(CSV_STRING.c_str());
  cam_s.flush();
  Serial.println(CSV_STRING);         // For Debugging purposes
  CSV_STRING.clear();
  intr = 0;
  packet++;
}
test();
if (chute_release == 1){
  if (first == 1){
  cam_s.write(take_pic);
  first = 0;
    }
  }
}
```

6.2) ESP32CAM:

```
#include "FS.h"
#include "SD_MMC.h"
```

```
#include "esp_camera.h"
#include <WiFi.h>
#include <ESPAsyncWebServer.h>
#define CAMERA_MODEL_AI_THINKER
#include "camera_pins.h"
TaskHandle_t listen_rx_handle;
TaskHandle_t take_image_handle;
TaskHandle_t transmit_image_handle;
TaskHandle_t log_handle;
TaskHandle_t test_acquire_image_handle;
TaskHandle_t warmup_handle;
const char* ssid = "CAM";
const char* password = "HAMZA154657";
byte rec = 0x00;
String rec_string ;
String pic = "/Image_0.jpg";
bool file_index = 0;                                  // Index for file; determines append or write
char pic_Index = '0';                                 // Index for picture
camera_fb_t *fb;
AsyncWebServer server(80);
void listen_rx(void *pvParameters) {                  // RTOSFunction for receive
  for (;;) {
    int current;
    int after;
    if (Serial.available()) {
      rec = Serial.read();
```

```
switch (rec) {
  case 'A':
    current = millis();
    xTaskNotifyGive(take_image_handle);
    after = millis();
    Serial.println(after - current);
    break;
  case 'B':
     xTaskCreatePinnedToCore(
       log_data, "log_" , 4096 ,
       NULL ,
       2 ,
       &log_handle,
       tskNO_AFFINITY );
    xTaskNotifyGive(log_handle);
    break;
  case 'Z':
    Serial.println("Transmit_called");
    xTaskCreatePinnedToCore(test_image, "testing_image",4096,
    NULL,
    2,
    &test_acquire_image_handle,
    1
    );
    break;
}
```

```
    }
    vTaskDelay(pdMS_TO_TICKS(1));
  }
}
void warmup(void *pvParameters){
int a = 0;
  while(a != 100){
    fb = esp_camera_fb_get();
    esp_camera_fb_return(fb);
    a++;
  }
  Serial.println("warmup_done");
  vTaskDelete(NULL);
}
void test_image(void *pvParameters){
  fb = esp_camera_fb_get();
  writeImage(SD_MMC ,"/TESTIMG.jpg", fb->buf,fb->len);
  esp_camera_fb_return(fb);
  vTaskDelete(NULL);
}
void take_image(void *pvParameters) {                    // RTOSFunction
for taking Images
  ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
  for (int a = 0; a < 5; a++) {
    for (int i = 0; i < 2; i++) {
      if (i < 2 - 1) {
```

```
      fb = esp_camera_fb_get();
      esp_camera_fb_return(fb);
    } else {
      fb = esp_camera_fb_get();
    }
  }
  if (!fb) {
    Serial.println("Error: Frame buffer could not be acquired");
  }
  switch (a) {
    case 0:
      writeImage(SD_MMC, pic.c_str(), fb->buf, fb->len);
      break;
    case 1:
      pic.replace(pic_Index - 1, pic_Index);
      writeImage(SD_MMC, pic.c_str(), fb->buf, fb->len);
      break;
    case 2:
      pic.replace(pic_Index - 1, pic_Index);
      writeImage(SD_MMC, pic.c_str(), fb->buf, fb->len);
      break;
    case 3:
      pic.replace(pic_Index - 1, pic_Index);
      writeImage(SD_MMC, pic.c_str(), fb->buf, fb->len);
      break;
    case 4:
```

```
    pic.replace(pic_Index - 1, pic_Index);
    writeImage(SD_MMC, pic.c_str(), fb->buf, fb->len);
    break;
  case 5:
    pic.replace(pic_Index - 1, pic_Index);
    writeImage(SD_MMC, pic.c_str(), fb->buf, fb->len);
    break;
  case 6:
    pic.replace(pic_Index - 1, pic_Index);
    writeImage(SD_MMC, pic.c_str(), fb->buf, fb->len);
    break;
  case 7:
    pic.replace(pic_Index - 1, pic_Index);
    writeImage(SD_MMC, pic.c_str(), fb->buf, fb->len);
    break;
  case 8:
    pic.replace(pic_Index - 1, pic_Index);
    writeImage(SD_MMC, pic.c_str(), fb->buf, fb->len);
    break;
  case 9:
    pic.replace(pic_Index - 1, pic_Index);
    writeImage(SD_MMC, pic.c_str(), fb->buf, fb->len);
    break;
  default:
    pic = "/Image_0.jpg";
    break;
```

```
    }
    //return the frame buffer back to be reused
    esp_camera_fb_return(fb);
    pic_Index++;
  }
  pic = "/Image_0.jpg";
  pic_Index = '0';
  vTaskDelete(NULL);
}
void log_data(void *pvParameters) {                                    // Function
for data logging
  ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
  for(;;){
  rec_string = Serial.readStringUntil('E');
  rec_string += '\n';
    appendFile(SD_MMC, "/log.txt", rec_string.c_str());
  break;
  }
  vTaskDelete(NULL);
}

void writeImage(fs::FS &fs, const char *path, const uint8_t *data, size_t len) {
  File file = fs.open(path, FILE_WRITE);
  if (!file) {
    return;
  }
```

```
  file.write(data, len);
  file.close();
}

void appendFile(fs::FS &fs, const char *path, const char *message) {                    // Function for appending to a file on SD_MMC
  Serial.printf("Appending to file: %s\n", path);

  File file = fs.open(path, FILE_APPEND);
  if (!file) {
    Serial.println("Failed to open file for appending");
    return;
  }
  if (file.print(message)) {

  } else {
    Serial.println("Append failed");
  }
  Serial.println("a done");
  file.close();
}
 //SETUP//
void setup() {
  Serial.begin(250000);
  camera_config_t config;  // Configure Camera
  config.ledc_channel = LEDC_CHANNEL_0;
```

```
config.ledc_timer = LEDC_TIMER_0;
config.pin_d0 = Y2_GPIO_NUM;
config.pin_d1 = Y3_GPIO_NUM;
config.pin_d2 = Y4_GPIO_NUM;
config.pin_d3 = Y5_GPIO_NUM;
config.pin_d4 = Y6_GPIO_NUM;
config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sccb_sda = SIOD_GPIO_NUM;
config.pin_sccb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.frame_size = FRAMESIZE_UXGA;
config.pixel_format = PIXFORMAT_JPEG;  // for streaming
config.grab_mode = CAMERA_GRAB_LATEST;
config.fb_location = CAMERA_FB_IN_PSRAM;
config.jpeg_quality = 10;
config.fb_count = 4;
esp_err_t err = esp_camera_init(&config);
while (err != ESP_OK) {
```

```
  Serial.println(esp_camera_init(&config));
  ESP.restart();
 }
 while (!SD_MMC.begin()) {
  Serial.println("SD init failed");
 }
 // Task creation should be at the end so that everything is initialized

 xTaskCreatePinnedToCore(                    // Create a task for listening to Serial on core 0
  listen_rx, "ListenRXfunc", 4096,           // Function, Name, Stackdepth
  NULL,                                      // Function parameters
  3,                                         // Priority
  &listen_rx_handle,                         // Handle
  0                                          // Core
 );
 xTaskCreatePinnedToCore(                  // Create a task for taking Image on core 1
  take_image, "Take_Imagefunc", 4096,
  NULL,
  2,
  &take_image_handle,
  1);
  xTaskCreatePinnedToCore(                  // Create a task for taking Image on core 1
  warmup, "Warmupfunc", 1024,
  NULL,
  3,
  &warmup_handle,
```

```
  1);

 // xTaskCreatePinnedToCore(                    // Create a task for transmitting images on core 1
 //   transmit_image, "transmitimgfunc", 2048,
 //   NULL,
 //   2,
 //   &transmit_image_handle,
 //   1);
}
```

6.3) Ground control system (GCS)

```
#include <AsyncEventSource.h>
#include <AsyncJson.h>
#include <AsyncWebSocket.h>
#include <AsyncWebSynchronization.h>
#include <ESPAsyncWebServer.h>
#include <WiFi.h>

const char* ssid = "MY_ESP";
const char* password = "hamzav@bod8w3g";
String received;
char *token;
String parsed[14];
int i = 0;
const char *a = ",";
int packet;
String Time;
```

```
float lat, lon, speed, temp, pressure, altitude;
float accel_x, accel_y, accel_z;
float gyro_x, gyro_y, gyro_z;

AsyncWebServer server(80);

void setup() {
  WiFi.softAP(ssid, password);
  Serial.begin(115200);

  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
    String html = R"rawliteral(
      <html>
      <head>
        <title>Parsed Data</title>
        <script>
          function fetchData(){
            fetch('/data')
              .then(response => response.json())
              .then(data => {
                document.getElementById('time').innerText = data.time;
                document.getElementById('packet').innerText = data.packet;
                document.getElementById('lat').innerText = data.lat;
                document.getElementById('lon').innerText = data.lon;
                document.getElementById('speed').innerText = data.speed;
                document.getElementById('temp').innerText = data.temp;
```

```
            document.getElementById('pressure').innerText = data.pressure;
            document.getElementById('alt').innerText = data.altitude;
            document.getElementById('a_x').innerText = data.accel_x;
            document.getElementById('a_y').innerText = data.accel_y;
            document.getElementById('a_z').innerText = data.accel_z;
            document.getElementById('g_x').innerText = data.gyro_x;
            document.getElementById('g_y').innerText = data.gyro_y;
            document.getElementById('g_z').innerText = data.gyro_z;
          })
          .catch(error => console.error('Error fetching data', error));
      }
      setInterval(fetchData, 250); // Fetches data every 1000 ms
    </script>
  </head>
  <body>
    <h1>Data:</h1>
    <div>
      <p>Time: <span id="time"></span></p>
      <p>Packet: <span id="packet"></span></p>
      <p>Latitude: <span id="lat"></span></p>
      <p>Longitude: <span id="lon"></span></p>
      <p>Speed: <span id="speed"></span></p>
      <p>Temperature: <span id="temp"></span></p>
      <p>Pressure: <span id="pressure"></span></p>
      <p>Altitude: <span id="alt"></span></p>
      <p>Acceleration_x: <span id="a_x"></span></p>
```

```
        <p>Acceleration_y: <span id="a_y"></span></p>
        <p>Acceleration_z: <span id="a_z"></span></p>
        <p>Gyro_x: <span id="g_x"></span></p>
        <p>Gyro_y: <span id="g_y"></span></p>
        <p>Gyro_z: <span id="g_z"></span></p>
      </div>
    </body>
    </html>
  )rawliteral";
  request->send(200, "text/html", html);
 });
 server.on("/data", HTTP_GET, [](AsyncWebServerRequest *request){
  String json = "{";
  json += "\"time\":\"" + Time + "\",";
  json += "\"packet\":" + String(packet) + ",";
  json += "\"lat\":" + String(lat) + ",";
  json += "\"lon\":" + String(lon) + ",";
  json += "\"speed\":" + String(speed) + ",";
  json += "\"temp\":" + String(temp) + ",";
  json += "\"pressure\":" + String(pressure) + ",";
  json += "\"altitude\":" + String(altitude) + ",";
  json += "\"accel_x\":" + String(accel_x) + ",";
  json += "\"accel_y\":" + String(accel_y) + ",";
  json += "\"accel_z\":" + String(accel_z) + ",";
  json += "\"gyro_x\":" + String(gyro_x) + ",";
  json += "\"gyro_y\":" + String(gyro_y) + ",";
```

```
    json += "\"gyro_z\":" + String(gyro_z);
    json += "}";
    request->send(200, "application/json", json);
  });

  server.begin();
}

void loop() {
  if (Serial.available()) {
    received = Serial.readStringUntil('\n');

    if (received.length() > 0) {
      char receivedarray[received.length() + 1];
      received.toCharArray(receivedarray, received.length() + 1);

      token = strtok(receivedarray, a);
      i = 0;
      while (token != NULL && i < 14) {
        parsed[i] = token;
        token = strtok(NULL, a);
        i++;
      }
      received = "";

      // Parsing Start
```

```
    if (i >= 14) {
      packet = parsed[0].toInt();
      Time = parsed[1];
      lat = parsed[2].toFloat();
      lon = parsed[3].toFloat();
      speed = parsed[4].toFloat();
      temp = parsed[5].toFloat();
      pressure = parsed[6].toFloat();
      altitude = parsed[7].toFloat();
      accel_x = parsed[8].toFloat();
      accel_y = parsed[9].toFloat();
      accel_z = parsed[10].toFloat();
      gyro_x = parsed[11].toFloat();
      gyro_y = parsed[12].toFloat();
      gyro_z = parsed[13].toFloat();
    }
    // Parsing End
  }
 }
}
```

6.3 Ground control system

```
#include <SPI.h>
#include <LoRa.h>
#define PIN_LORA_COPI   23
#define PIN_LORA_CIPO   19
#define PIN_LORA_SCK    18
```

```
#define PIN_LORA_CS     5
#define PIN_LORA_RST    2
#define PIN_LORA_DIO0   4

#define LORA_FREQUENCY  433E6
void setup() {
  Serial.begin (115200);
  LoRa.setPins (PIN_LORA_CS, PIN_LORA_RST, PIN_LORA_DIO0);
  LoRa.setSPIFrequency (20000000);
  LoRa.begin(LORA_FREQUENCY);
  delay(1000);
  while (!LoRa.begin (LORA_FREQUENCY)) {
    Serial.println ("Starting LoRa failed");
    Serial.println (LoRa.begin (LORA_FREQUENCY));
    //ESP.restart();
  }
}
void loop() {
  // try to parse packet
  int packetSize = LoRa.parsePacket();

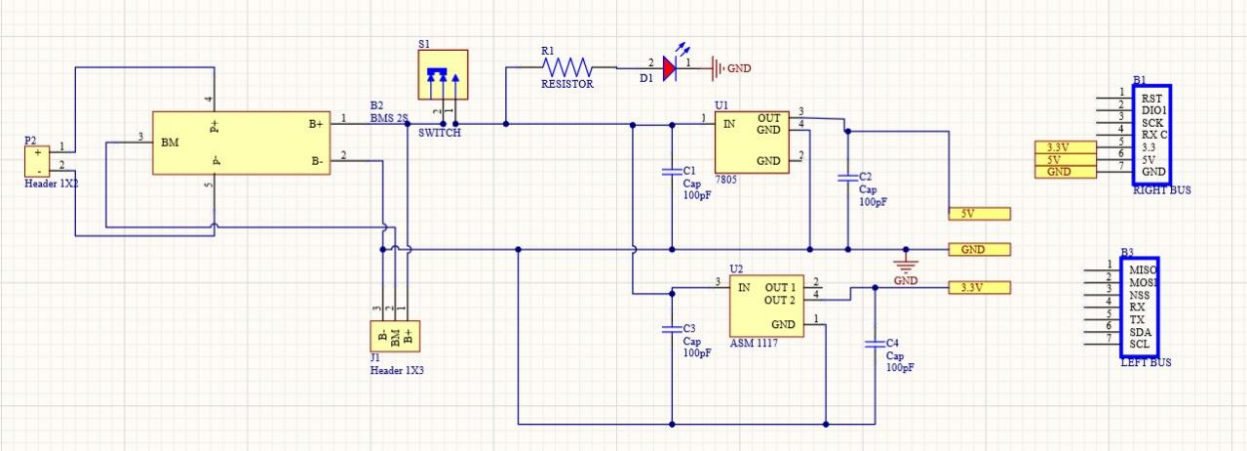
  if (packetSize) {
    // read packet
    while (LoRa.available()) {
      Serial.print ((char) LoRa.read());
    }
```

```
  // print RSSI of packet
  Serial.print ("\n");
  //Serial.println (LoRa.packetRssi());
 }
```

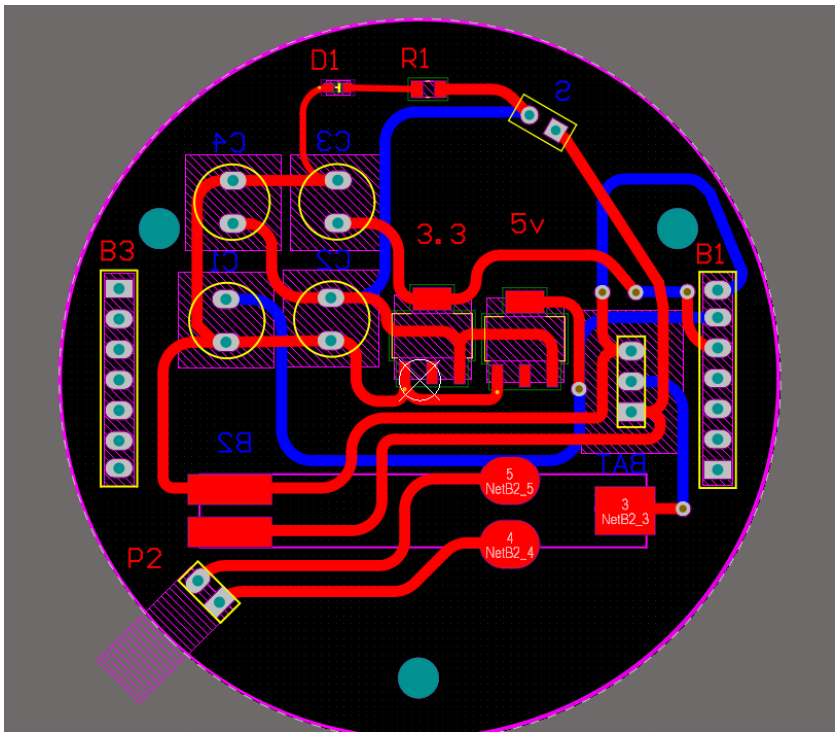
Pcb design Circuitry

Power system

Schematics design

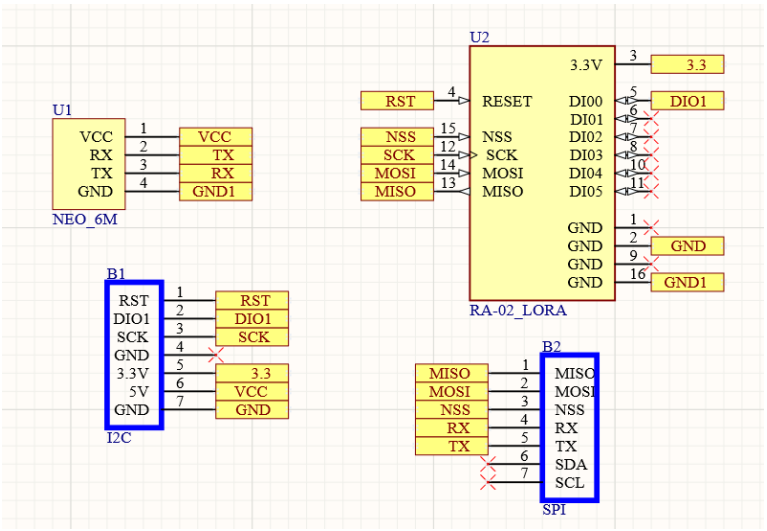


Pcb layout

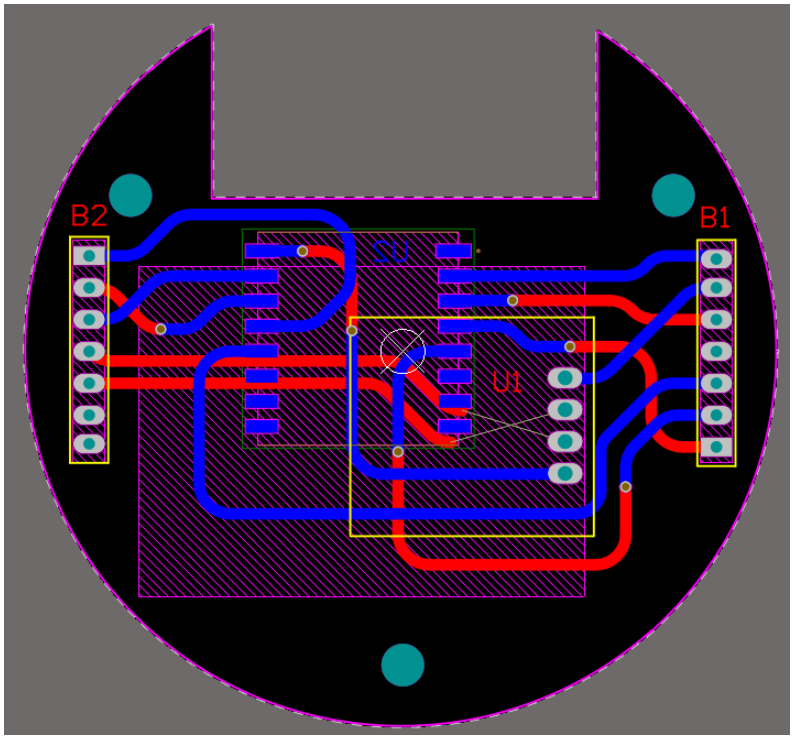

D1
R1
S
C4
C3
3.3
5v
B3
B1
C1
C2
B2
BAT
5
NetB2_5
3
NetB2_3
4
NetB2_4
P2

communication system

schematics design

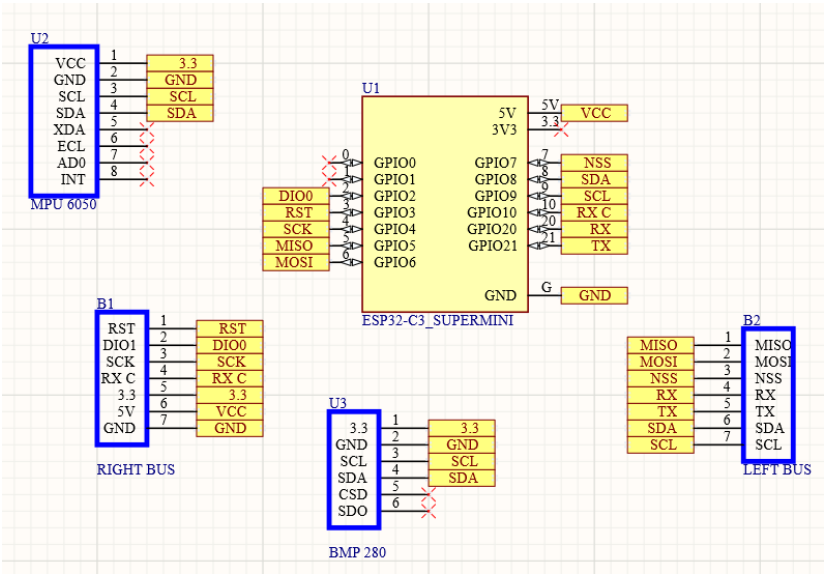


pcb design

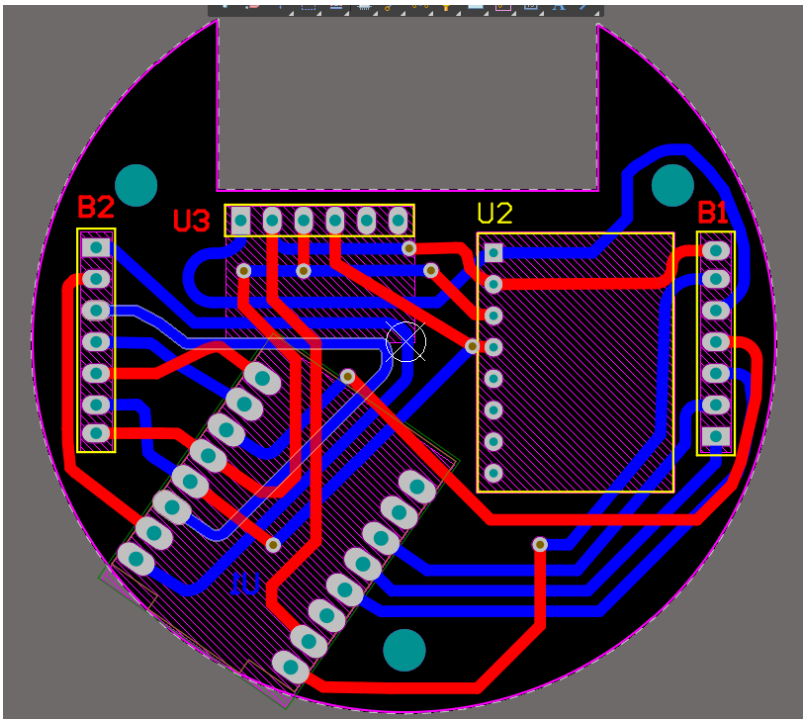


control and sensor system

schematics

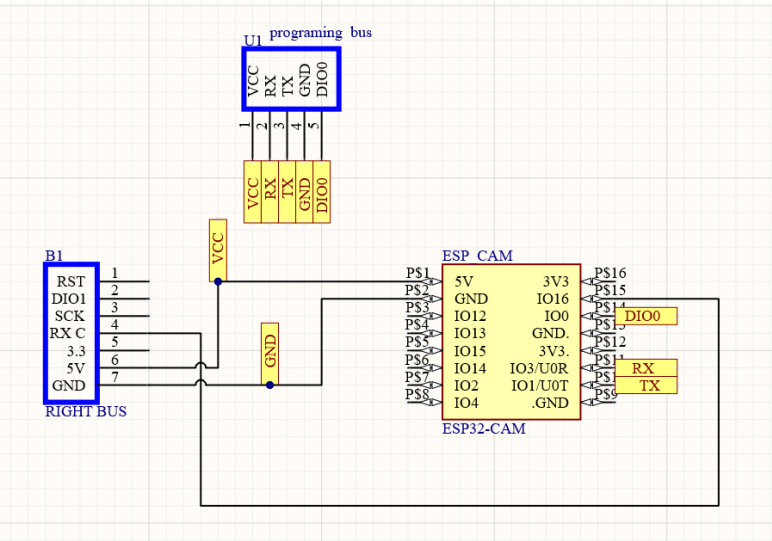


Pcb design



Camera or payload system

Schematics



Pcb design

